

CS112: Wednesday 7 October, 12:15 to 1:20

Last class we turned `PyList` into `PyList<Item>`. We removed the `typedef` from the code, and updated each method with a new header and moved the code to `PyList.h`. We also tested our class adding students names to a `PyList<string>` `names`.

Today: `vector` from the standard library, and the iterators that walk it. We close on what operations cost. Indexing is constant, the append we wrote is amortized constant, and an append that grows by one slot is linear.

PART

Daily quiz

1. `vector`, which you have already written
2. `begin()`, `end()`, and one past the last
3. `find` over a pair of iterators
4. What indexing costs
5. What `append` costs
6. Close

Part 1. `vector`, which you have already written

Open `w05-vectors/main.cpp`. Four names pushed back, one assertion on `stds[3]`, one on `size()`. It runs.

```
vector<string> stds;  
stds.push_back("Micaiah");  
assert(stds[3] == "Diego");  
assert(stds.size() == 4);
```

Talk to your neighbor. Line by line, what does our `PyList` call each of these?

`push_back` is `append`. `size()` is `getSize()`. `operator[]` is the one we wrote twice. There is no `getCapacity()` in the assertions, and `capacity()` exists.

Two differences, on the board:

|| `PyList` | `vector` | |---| | `v[99]` out of range | throws
`invalid_argument` | reads whatever is there | | checked access |
`getIndex` | `at()` |

`PyList` throws on a bad index. `vector` does not check `[]` at all, and `at()` is where its check lives.

Part 2. `begin()`, `end()`, and one past the last

```
stds.begin()    // the position of stds[0]
stds.end()      // the position AFTER stds[3]
```

An iterator is a position, not a value. `*it` is the value at it.

Talk to your neighbor. `end()` points at something nobody may read. Why have it at all?

Two answers, both on the board:

```
vector<string>::iterator it;
for (it = stds.begin(); it != stds.end(); ++it)
    cout << *it << " ";           // the range is [begin, end)

if (stds.begin() == stds.end())
    cout << "empty";              // no special case needed
```

The loop stops at `end()` without reading it. On an empty vector `begin()` is already `end()`, so the loop body runs zero times with no test written for it.

Part 3. `find` over a pair of iterators

```
#include <algorithm>

vector<string>::iterator it = find(stds.begin(), stds.end(),
    "Diego");

if (it == stds.end()) cout << "not here\n";
else cout << *it << " at " << (it - stds.begin());
```

Why does `find` take two iterators instead of the vector?

Two positions are enough to say what to search. The same `find` then works on half a vector, on an array, and on a container nobody has written yet.

What does `find` return when it fails, and why that?

`end()`. That position always exists and is never an element, so failure needs no special value of its own.

Run it. Then run it again with `"Kwesi"`.

Part 4. What indexing costs

```
stds[0]
stds[3]
stds[499999]
```

Talk to your neighbor. Which of those three takes longest?

None of them. One multiply and one add, from the same arithmetic as before: `base + sizeof(Item) * index`. The size of the vector never enters it.

$$\text{address}(v[i]) = \text{base} + \text{sizeof}(\text{Item}) \times i$$

Indexing is a **constant time** operation.

A contiguous block is what makes that arithmetic work, and it is also why `push_back` moves everything when it grows. The block has a fixed size, so a bigger one is a different block.

Part 5. What `append` costs

Two cases, side by side on the board.

```
myArray[mySize] = it;    // room left: 1 store
++mySize;               //           1 add
```

```
new Item[2 * n]          // full: room for 2n items
copy n items             //           3 operations each
delete [] old
myArray = bigger
```

Two operations against $3n + 5$, where the 5 is the work that does not depend on n : the allocate, the store, the delete, the repoint and the count.

So which is it?

The chart, one append at a time. Sixteen appends, one block each for storing the item, and a stack of copies on the four that found the array full: 1, 2, 4 and 8. The capacity bracket underneath doubles as the array does, so the expensive ones get further apart.

Then the same blocks laid flat, two rows high. 31 blocks over 16 appends.

The algebra behind the picture:

$$\begin{array}{lcl} \text{n appends after the array fills:} & n + (n - 1) * 1 = & 2n - 1 \\ \text{per append:} & (2n - 1) / n = & 2 - 1/n \end{array}$$

As n grows, $1/n$ shrinks and the average settles under 2. It does not grow with n , which is what **amortized constant** means.

Talk to your neighbor. An earlier `append` grew the array by one slot each time it filled up. What did one append cost then?

Every append finds the array full, so every one of them allocates, copies all n items and frees. All of them are expensive, and there is nothing cheap for the cost to be spread across. That one is **linear**.

Talk to your neighbor. Inserting at the front of a dynamic array is...

Every element already there moves one slot first, and the last one moves first of all. **Linear** again.

Part 6. Close

Three costs so far: indexing is constant, `append` is amortized constant, inserting at the front is linear.

Friday: Prof. Victor Norman.